

GPU Systems and Infrastructure

System Validation

Contents

[Print to PDF](#) ▶

- RVS Validation
- Performance Benchmarking
- AGFHC Validation

This document outlines the procedures for validating AMD Instinct™ GPU systems. It covers [RVS](#), [AGFHC](#) and additional validation tools.

RVS Validation

The validation tests in this section are intended to ensure that a system is operating correctly. In this section, ROCm Validation Suite (`rvs`) is used, which is a collection of tests, benchmarks, and qualification tools, each targeting a specific subsystem of the system under test (SUT).

If not already installed on the SUT, run the following install command (Ubuntu):

```
sudo apt install rocm-validation-suite
```

Then ensure that the path to the `rvs` executable, located at `/opt/rocm/bin`, is added to the path. Use the following command:

```
export PATH=$PATH:/opt/rocm/bin
```

The `rvs` tool consists of modules that implement a particular test functionality. The collection of the modules can be broadly categorized as targeting the following aspects of the hardware platform:

- [Compute / GPU](#)
- [Memory](#)
- [IO / PCIe](#)

Each of these categories runs a subset of `rvs` modules to validate that the category is working as expected.

The standard way to run an `rvs` module is by providing a configuration file prefixed with the `-c` option. When `rvs` is installed properly on the SUT, the `conf` files are found in the `/opt/rocm/share/rocm-validation-suite/conf/` folder. Since this path is a part of every `rvs` command in this document, an environment variable is defined which will be used in place of the long path for commands and their output. To set this variable in the environment, run the following command:

```
export RVS_CONF=/opt/rocm/share/rocm-validation-suite/conf
```

The configuration files section of the ROCm Validation Suite User Guide (<https://github.com/ROCm/ROCmValidationSuite/blob/master/docs/ug1main.md#configuration-files>) provides detailed description about the `conf` file, its formation, and keys. It's recommended to become familiar with the `conf` file before running the `rvs` tests described in the following sections. Be aware that some `conf` files are included in product-specific subfolders (for instance, `/opt/rocm/share/rocm-validation-suite/conf/MI300X`). If present, always use GPU-specific configurations instead of the default test configurations.

In the following subsections, under each of the categories, the relevant `rvs` test modules are listed along with descriptions how the category is validated. Example `rvs` commands with the expected output are also provided. Most of the `rvs` tests do not have strict PASS / FAIL conditions reported; rather it is expected that when they are run on the SUT, the output observed are within a reasonable range provided.

Compute / GPU

The `rvs` has two different types of modules to validate the Compute subsystem. These are:

- [Properties](#)
- [Benchmark / Stress / Qualification](#)

MI300X GPU accelerators have many architectural features. Similar to the [Check GPU presence \(lspci\)](#) section, `rvs` has an option to display all MI300X GPU accelerators present in the SUT. Before proceeding with the modules below, run the following command to make sure all the GPUs are seen with their correct PCIe properties.

Command:

```
rvs -g
```

Expected output:

```
ROCm Validation Suite (version 0.0.60202)
```

```
Supported GPUs available:
```

```
0000:05:00.0 - GPU [ 2 - 28851] AMD Instinct MI300X (Device 29857)
0000:26:00.0 - GPU [ 3 - 23018] AMD Instinct MI300X (Device 29857)
0000:46:00.0 - GPU [ 4 - 29122] AMD Instinct MI300X (Device 29857)
0000:65:00.0 - GPU [ 5 - 22683] AMD Instinct MI300X (Device 29857)
0000:85:00.0 - GPU [ 6 - 53458] AMD Instinct MI300X (Device 29857)
0000:a6:00.0 - GPU [ 7 - 63883] AMD Instinct MI300X (Device 29857)
0000:c6:00.0 - GPU [ 8 - 53667] AMD Instinct MI300X (Device 29857)
0000:e5:00.0 - GPU [ 9 - 63738] AMD Instinct MI300X (Device 29857)
```

Result:

- PASSED: All 8 GPUs are seen in the output
- FAILED: Otherwise
 - Action: Don't proceed further. Debug the issue of not being able to see all GPUs.

GPU Properties

The GPU Properties module queries the configuration of a targeted GPU and returns the device's static characteristics. These static values can be used to debug issues such as device support, performance and firmware problems.

To confirm the architectural properties of the GPU, use the GPUP module, which uses the GPUP configuration file.

The configuration file for GPUP module is located at `${RVS_CONF}/gpup_single.conf`.

The GPUP module section of the ROCm Validation Suite User Guide

(<https://github.com/ROCm/ROCmValidationSuite/blob/master/docs/ug1main.md#gpup-module>) provides detailed description about the GPUP `conf` file, its formation, and keys.

Command:

```
rvs -c ${RVS_CONF}/gpup_single.conf
```

Expected output (truncated):

The `conf` file has six test cases `RVS-GPUP-TC1`, `RVS-GPUP-TC2`, and so on up to `RV-GPUP-TC6`. Only a truncated version of the output of `RVS-GPUP-TC1` is shown here. The other tests are modified versions of `RVS-GPUP-TC1`, which display a subset of properties and/or a subset of `io_links-properties`.

The first block of output displays the properties (all):

```
[RESULT] [ 54433.732433] Action name :RVS-GPUP-TC1
[RESULT] [ 54433.733858] Module name :gpup
[RESULT] [ 54433.733992] [RVS-GPUP-TC1] gpup 28851 cpu_cores_count 0
[RESULT] [ 54433.733994] [RVS-GPUP-TC1] gpup 28851 simd_count 1216
...
[RESULT] [ 54433.734018] [RVS-GPUP-TC1] gpup 28851 num_xcc 8
[RESULT] [ 54433.734018] [RVS-GPUP-TC1] gpup 28851 max_engine_clk_ccompute 3250
```

The block below shows only one of the `io_link-properties` of the eight GPUs (`0` to `7`):

```
[RESULT] [ 96878.647964] [RVS-GPUP-TC1] gpup 28851 0 type 2
[RESULT] [ 96878.647973] [RVS-GPUP-TC1] gpup 28851 0 version_major 0
[RESULT] [ 96878.647982] [RVS-GPUP-TC1] gpup 28851 0 version_minor 0
[RESULT] [ 96878.647990] [RVS-GPUP-TC1] gpup 28851 0 node_from 2
[RESULT] [ 96878.647997] [RVS-GPUP-TC1] gpup 28851 0 node_to 0
[RESULT] [ 96878.648013] [RVS-GPUP-TC1] gpup 28851 0 weight 20
[RESULT] [ 96878.648020] [RVS-GPUP-TC1] gpup 28851 0 min_latency 0
[RESULT] [ 96878.648029] [RVS-GPUP-TC1] gpup 28851 0 max_latency 0
[RESULT] [ 96878.648037] [RVS-GPUP-TC1] gpup 28851 0 min_bandwidth 312
[RESULT] [ 96878.648045] [RVS-GPUP-TC1] gpup 28851 0 max_bandwidth 64000
[RESULT] [ 96878.648053] [RVS-GPUP-TC1] gpup 28851 0 recommended_transfer_size
[RESULT] [ 96878.648060] [RVS-GPUP-TC1] gpup 28851 0 flags 1
```

Result:

- PASSED: If generated output looks similar
- FAILED: If any GPU is not listed in output or ERROR tagged logs are seen
 - Typically, it is not expected that this module will fail

Benchmark, stress, qualification

These categories of modules perform qualification of the GPU subsystem, execute stress test, and compute and display bandwidth. The modules do not produce a PASS / FAIL result. When bandwidth is measured, it only reports the bandwidth and doesn't make any comparisons with the existing set of numbers. The only exceptions are GST and IET modules.

GPU Stress Test

The GPU Stress Test (GST) module stresses the GPU FLOPS performance for SGEMM, DGEMM and HGEMM operations and computes and displays peak GFLOPs/s. Two configuration files are used by the GST module – one is general purpose (`gst_single.conf`), and the other is MI300X specific (`gst_ext.conf`). Each is detailed below.

The MI300X specific `gst_single.conf` configuration file for the GST module is located at:

```
${RVS_CONF}/MI300X/gst_single.conf
```

Run the following command to perform the general GPU stress test using the `gst_single.conf` config file.

```
rvs -c ${RVS_CONF}/MI300X/gst_single.conf
```

Expected output (truncated):

```
[RESULT] [1101980.682169] Action name :gst-1215Tflops-4K4K8K-rand-fp8
[RESULT] [1101980.683973] Module name :gst
[RESULT] [1101980.836841] [gst-1215Tflops-4K4K8K-rand-fp8] [GPU:: 28851] Start
[RESULT] [1101987.830800] [gst-1215Tflops-4K4K8K-rand-fp8] [GPU:: 28851] GFLOPS
[RESULT] [1101988.831928] [gst-1215Tflops-4K4K8K-rand-fp8] [GPU:: 28851] End of
[RESULT] [1101992.16545 ] [gst-1215Tflops-4K4K8K-rand-fp8] [GPU:: 28851] GFLOPS
...
[RESULT] [1102003.864648] [gst-1215Tflops-4K4K8K-rand-fp8] [GPU:: 28851] GFLOPS
```

Result:

- PASSED: If `met: TRUE` is displayed in test log for all eight GPUs and actions, it indicates the test was able to hit peak GFLOP/s which matches or exceeds the target values listed in the config file.
- FAILED: Test results fail to meet the target GFLOP/s
 - Action: Do not proceed further. Report this issue to your system manufacturer immediately.

The MI300X specific `gst_ext.conf` configuration file for the GST module is located at:

```
${RVS_CONF}/MI300X/gst_ext.conf
```

Run the following command to perform the MI300X GPU specific stress test using the

`gst_ext.conf` config file.

```
ROCBLAS_TENSILE_LIBPATH=/opt/rocm/lib/rocblas/library/ rvs -c ${RVS_CONF}/MI300
```

Expected output (truncated):

```
[RESULT] [603545.521766] Action name :gst-1000Tflops-8KB-fp8_r-false
[RESULT] [603545.523245] Module name :gst
[RESULT] [603545.685745] [gst-1000Tflops-8KB-fp8_r-false] [GPU:: 28851] Start o
[RESULT] [603552.11787 ] [gst-1000Tflops-8KB-fp8_r-false] [GPU:: 28851] GFLOPS
...
[RESULT] [603585.12457 ] [gst-1000Tflops-8KB-fp8_r-false] [GPU:: 28851] GFLOPS
```

Result:

- PASSED: If "met: TRUE" is displayed in the test log for all eight GPUs, it indicates the test was able to hit peak GFLOP/s which matches or exceeds the target values listed in the config file.
- FAILED: Test results fail to meet the target GFLOP/s
 - Action: Do not proceed further. Report this issue to your system manufacturer immediately.

Input Energy Delay Product

The Input Energy Delay Product (EDP) test (IET) module runs GEMM workloads to stress the GPU power, that is, Total Graphics Power (TGP).

This test is used to:

- Verify the GPU can handle maximum power stress for a sustained period.
- Check that the GPU power reaches a set target power.

The configuration file for IET module is located at `${RVS_CONF}/MI300X/iet_single.conf`.

Command:

```
rvs -c ${RVS_CONF}/MI300X/iet_single.conf
```

IET module run six different actions. Each action will be performed on all eight GPUs. Each GPU power test will display a `TRUE` or `FALSE` status as shown in the following output example.

Expected output (truncated):

```
[RESULT] [1102597.157090] Action name :iet-620W-1K-rand-dgemm
[RESULT] [1102597.159274] Module name :iet
[RESULT] [1102597.333747] [iet-620W-1K-rand-dgemm] [GPU:: 28851] Power(W) 127.0
[RESULT] [1102597.334457] [iet-620W-1K-rand-dgemm] [GPU:: 23018] Power(W) 123.0
...
[RESULT] [1102657.372824] [iet-620W-1K-rand-dgemm] [GPU:: 29122] pass: TRUE
[RESULT] [1102657.372859] [iet-620W-1K-rand-dgemm] [GPU:: 23018] pass: TRUE
[RESULT] [1102657.372936] [iet-620W-1K-rand-dgemm] [GPU:: 28851] pass: TRUE
...
[RESULT] [1102658.379728] Action name :iet-wait-750W-28K-rand-dgemm
[RESULT] [1102658.379781] Module name :iet
```

Result:

- PASSED: `pass: TRUE` must be displayed for each GPU.
- FAILED: Test results FAIL
 - Action: Do not proceed further. Report this issue to your system manufacturer immediately.

Qualification

The GPU monitor (GM) module is used to report and validate the following system attributes.

- Temperature
- Fan speed
- Memory clock
- System clock
- Power

The configuration file for GST module is located at `${RVS_CONF}/gm_single.conf`.

Command:

```
rvs -c ${RVS_CONF}/gm_single.conf
```

Expected output (truncated):

```
[RESULT] [209228.305186] [metrics_monitor] gm 28851 temp violations 0
[RESULT] [209228.305186] [metrics_monitor] gm 28851 clock violations 0
[RESULT] [209228.305186] [metrics_monitor] gm 28851 mem_clock violations 0
[RESULT] [209228.305186] [metrics_monitor] gm 28851 fan violations 0
[RESULT] [209228.305186] [metrics_monitor] gm 28851 power violations 0
...
```

Result:

- PASSED: If the output displays `violations 0` for all give attributes for each GPU. Pipe output to grep to create a quick summary of violations.
- FAILED: If any violations have a non-zero value
 - Action: Continue with the next step but periodically monitor by running this module.

Memory

To validate the GPU memory subsystem, `rvs` has the following two *types* of modules:

- [MEM](#)
- [BABEL](#)

MEM

The Memory module, MEM, tests the GPU memory for hardware errors and soft errors using HIP. It consists of various tests that use algorithms like Walking 1 bit, Moving inversion and Modulo 20. The module executes the following memory tests [Algorithm, data pattern]:

- Walking 1 bit
- Own address test
- Moving inversions, ones & zeros
- Moving inversions, 8-bit pattern
- Moving inversions, random pattern
- Block move, 64 moves
- Moving inversions, 32-bit pattern
- Random number sequence
- Modulo 20, random pattern
- Memory stress test

The configuration file for GST module is located at `${RVS_CONF}/mem.conf`.

Command:

```
rvs -c ${RVS_CONF}/mem.conf -l mem.txt
```

The entire output file is not shown here for brevity. Grepping for certain strings in the file where the log is saved makes it easier to understand the log. The `-l mem.txt` option in the command dumps the entire output into the file.

Grepping for the string `mem Test 1:` shows, Test 1 (Change one bit memory address) is launched for each GPU.

```
grep "mem Test 1:" mem.txt
[RESULT] [214775.925788] [action_1] mem Test 1: Change one bit memory addresss
[RESULT] [214776.112738] [action_1] mem Test 1: Change one bit memory addresss
[RESULT] [214776.299030] [action_1] mem Test 1: Change one bit memory addresss
[RESULT] [214776.486354] [action_1] mem Test 1: Change one bit memory addresss
[RESULT] [214776.674529] [action_1] mem Test 1: Change one bit memory addresss
[RESULT] [214776.865057] [action_1] mem Test 1: Change one bit memory addresss
[RESULT] [214777.52685 ] [action_1] mem Test 1: Change one bit memory addresss
[RESULT] [214777.155703] [action_1] mem Test 1: Change one bit memory addresss
```

Grepping for the string `mem Test 1 :` shows, Test 1 passed for all GPUs.

```
[RESULT] [214775.947349] [action_1] mem Test 1 : PASS
[RESULT] [214776.134798] [action_1] mem Test 1 : PASS
[RESULT] [214776.320838] [action_1] mem Test 1 : PASS
[RESULT] [214776.509205] [action_1] mem Test 1 : PASS
[RESULT] [214776.697979] [action_1] mem Test 1 : PASS
[RESULT] [214776.888054] [action_1] mem Test 1 : PASS
[RESULT] [214777.75572 ] [action_1] mem Test 1 : PASS
[RESULT] [214777.178653] [action_1] mem Test 1 : PASS
```

Similarly, you can grep other strings to parse the log file easily.

Grepping for the string "bandwidth" shows the memory bandwidth perceived by each of the eight GPUs.

```
grep "bandwidth" mem.txt
[RESULT] [214808.291036] [action_1] mem Test 11: elapsedtime = 6390.423828 band
[RESULT] [214812.175895] [action_1] mem Test 11: elapsedtime = 6387.198242 band
[RESULT] [214813.999085] [action_1] mem Test 11: elapsedtime = 6400.554199 band
[RESULT] [214814.406234] [action_1] mem Test 11: elapsedtime = 6397.101074 band
[RESULT] [214814.583630] [action_1] mem Test 11: elapsedtime = 6388.572266 band
[RESULT] [214815.176800] [action_1] mem Test 11: elapsedtime = 6378.345703 band
[RESULT] [214815.384878] [action_1] mem Test 11: elapsedtime = 6404.943848 band
[RESULT] [214815.419048] [action_1] mem Test 11: elapsedtime = 6416.849121 band
```

Result:

- PASSED: If all memory tests passed without memory errors and the bandwidth obtained in Test 11 is about ~2TB/s
- FAILED: If any memory errors report and/or the obtained bandwidth is not even close to 2TB/s
 - Action: Do not proceed further. Report this issue to your system manufacturer immediately.

BABEL

Refer to the [BabelStream section](#) for instructions on how to run this module to test memory.

IO

To validate the GPU interfaces, `rvs` has the following three *types* of modules:

- [PEBB](#) – PCIe Bandwidth Benchmark
- [PEQT](#) – PCIe Qualification Tool
- [PBQT](#) – P2P Benchmark and Qualification Tool

PCIe Bandwidth Benchmark

The PCIe Bandwidth Benchmark attempts to saturate the PCIe bus with DMA transfers between system memory and a target GPU card's memory. The maximum bandwidth obtained is reported.

The configuration file for GST module is located at:

```
{RVS_CONF}/MI300X/pebb_single.conf
```

Command:

```
rvs -c ${RVS_CONF}/MI300X/pebb_single.conf -l pebb.txt
```

The PEBB modules has the following tests defined in the `conf` file (where `h2d` means host to device, `d2h` means device to host, `xMB` means random block size, and `b2b` means back to back):

- `h2d-sequential-51MB`
- `d2h-sequential-51MB`
- `h2d-d2h-sequential-51MB`
- `h2d-parallel-xMB`
- `d2h-parallel-xMB`
- `h2d-d2h-xMB`
- `h2d-b2b-51MB`
- `d2h-b2b-51MB`
- `h2d-d2h-b2b-51MB`

Each of these tests will produce the following header as part of the output log. It shows the distances between CPUs and GPUs.

Expected output (truncated):

```
[RESULT] [1103843.610745] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 0] [GPU::
[RESULT] [1103843.610763] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 1] [GPU::
[RESULT] [1103843.610771] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 0] [GPU::
[RESULT] [1103843.610778] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 1] [GPU::
[RESULT] [1103843.610787] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 0] [GPU::
[RESULT] [1103843.610795] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 1] [GPU::
[RESULT] [1103843.610802] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 0] [GPU::
[RESULT] [1103843.610810] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 1] [GPU::
[RESULT] [1103843.610817] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 0] [GPU::
[RESULT] [1103843.610825] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 1] [GPU::
[RESULT] [1103843.610833] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 0] [GPU::
[RESULT] [1103843.610841] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 1] [GPU::
[RESULT] [1103843.610848] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 0] [GPU::
[RESULT] [1103843.610856] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 1] [GPU::
[RESULT] [1103843.610863] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 0] [GPU::
[RESULT] [1103843.610871] [d2h-sequential-64MB] pcie-bandwidth [CPU:: 1] [GPU::
```

The other half of the output for each of the tests, shows the transfer bandwidth and indicates whether its bidirectional or unidirectional transfer.

```
[RESULT] [1103903.617888] [d2h-sequential-64MB] pcie-bandwidth [ 1/16] [CPU:: 0]
[RESULT] [1103903.617971] [d2h-sequential-64MB] pcie-bandwidth [ 2/16] [CPU:: 1]
...
[RESULT] [1103903.618142] [d2h-sequential-64MB] pcie-bandwidth [16/16] [CPU:: 1]
```

Result:

- PASSED: If all CPUs-GPUs distances are displayed and CPU x (x=0/1) to GPU y (y=2/3/4/5/6/7/8/9) PCIe transfer bandwidths are displayed.
- FAILED: Otherwise
 - Action: Proceed to next step. Run this same test later again.

PCIe Qualification Tool

The PCIe Qualification Tool is used to qualify the PCIe bus the GPU is connected to. The qualification tool can determine the following characteristics of the PCIe bus interconnect to a GPU:

- Support for Gen 3 atomic completers
- DMA transfer statistics
- PCIe link speed
- PCIe link width

The configuration file for the PEQT module is located at `${RVS_CONF}/peqt_single.conf`.

Command:

```
sudo rvs -c ${RVS_CONF}/peqt_single.conf
```

This module has total 17 tests (`pcie_act_1` to `pcie_act_17`). Each test checks for a subset of PCIe capabilities and shows the true or false status.

Note

The tests need `sudo` permission to run properly.

Expected output:

```
[RESULT] [1105558.986882] Action name :pcie_act_1
[RESULT] [1105558.988288] Module name :peqt
[RESULT] [1105559.33461 ] [pcie_act_1] peqt true
[RESULT] [1105559.33492 ] Action name :pcie_act_2
...
[RESULT] [1105559.651564] [pcie_act_17] peqt true
```

Result:

- PASSED: `[pcie_act_x] peqt true` can be seen for all 17 actions.
- FAILED: If any tests show true.
 - Action: Check that you are running this test as root or with `sudo` privileges. If not, actions 6 through 16 will fail. Run this same test later again.

P2P Benchmark and Qualification Tool

The PBQT module executes the following tests:

- List all GPUs that support P2P
- Characterizes the P2P links between peers
- Performs a peer-to-peer throughput test between all P2P pairs

The configuration file for the `pbqt` module for MI300X is located here:

```
${RVS_CONF}/MI300X/pbqt_single.conf
```

The `conf` file has 12 `actions_xy` test segments. Each of these checks for peer-to-peer connectivity among GPUs and provides a true/false status. In addition, it also performs bidirectional throughput test and reports the throughput obtained based on config parameters. Since comparison is not performed for some target throughput numbers, there is no PASS/FAIL condition for the overall test.

It's recommended that you carefully review the `pbqt_single.conf` file before running the following command.

Command:

```
rvs -c ${RVS_CONF}/MI300X/pbqt_single.conf
```

Only two example lines from the very long log file is shown because other lines look similar as all combinations of GPU pairs are considered and numbers for those pairs are reported.

Expected output below (truncated) shows uni-directional connectivity is true for the GPU and its connection to the other seven GPU peers:

```
[RESULT] [1104553.34268 ] [p2p-unidir-sequential-64MB] p2p [GPU:: 2 - 28851 - 0
[RESULT] [1104553.34276 ] [p2p-unidir-sequential-64MB] p2p [GPU:: 2 - 28851 - 0
...
```

The following lines show unidirectional throughput between the 56 GPU pairs (not all are shown):

```
[RESULT] [1104673.143726] [p2p-unidir-parallel-64MB] p2p-bandwidth[ 1/56] [GPU:
[RESULT] [1104673.144823] [p2p-unidir-parallel-64MB] p2p-bandwidth[ 2/56] [GPU:
[RESULT] [1104673.145898] [p2p-unidir-parallel-64MB] p2p-bandwidth[ 3/56] [GPU:
```

Result:

- PASSED: If `peers:true` lines are observed for GPUs peer-to-peer connectivity and if throughput values are non-zeros.
 - FAILED: Otherwise
 - Action: Do not proceed further. Report this issue to your system manufacturer immediately.
-

Performance Benchmarking

After completing the functional RVS validation (presence, health, memory, PCIe, and P2P), run the following performance benchmarks to confirm the system delivers expected compute, communication, and memory bandwidth. Full instructions and context are in the dedicated benchmarking guide (`../mi300x/performance-bench.md`). This section summarizes the required tests, example commands, and pass/fail criteria so results can be recorded as part of system acceptance.

Summary of Required Benchmarks

Area	Tool / Test	Command (example)	Pass / Fail Threshold (per guide)
GPU ↔ GPU & GPU ↔ CPU copies	TransferBench a2a	<code>TransferBench a2a</code>	Avg bandwidth ≥ 32.9 GB/s
GPU ↔ GPU (p2p)	TransferBench p2p	<code>TransferBench p2p</code>	Uni ≥ 33.9 GB/s; Bi ≥ 43.9 GB/s
Mixed copy patterns	TransferBench example.cfg Test 1–6	<code>TransferBench examples/example.cfg</code>	T1 47.1, T2 48.4, T3 (0→1) 31.9, T3 (1→0) 38.9, T4 1264, T6 48.6 GB/s (T5 N/A)
Collective bus BW	RCCL all_reduce	<code>build/all_reduce_perf -b 8 -e 8G -f 2 -g 8</code>	8-GPU all_reduce bus BW ≥ 304 GB/s
GEMM FP32	rocBLAS FP32 GEMM	<code>rocblas-bench -f gemm -r s -m 4000 -n 4000 -k 4000 --lda 4000 --ldb 4000 --ldc 4000 --transposeA N --transposeB T</code>	≥ 94,100 TFLOPS (peak achieved in any run)
GEMM BF16 (strided batched)	rocBLAS BF16	<code>rocblas-bench -f gemm_strided_batched_ex ... --a_type h ...</code> (see below)	≥ 130,600 TFLOPS
GEMM INT8 (strided batched)	rocBLAS INT8	<code>rocblas-bench -f gemm_strided_batched_ex ... --a_type i8_r ...</code>	≥ 162,700 TFLOPS

Area	Tool / Test	Command (example)	Pass / Fail Threshold (per guide)
Memory STREAM- like ops	BabelStream Triad, etc. (8 GPUs)	<code>mpiexec -n 8 wrapper.sh</code>	Copy 4,177,285; Mul 4,067,069; Add 3,920,853; Triad 3,885,301; Dot 3,660,781 MB/s

Note

rocBLAS performance can vary between runs; use the best (stable) value after several iterations.

TransferBench

Validate intra-node GPU copy performance and CPU↔GPU pathways.

Prerequisites: ROCm + `libnuma`.

Build (if not already installed):

```
git clone https://github.com/ROCm/TransferBench.git
cd TransferBench
CC=hipcc make
```

Run tests:

- All-to-all: `TransferBench a2a` (pass if avg $\geq$ 32.9 GB/s)
- Peer-to-peer: `TransferBench p2p` (pass if uni $\geq$ 33.9 GB/s & bi $\geq$ 43.9 GB/s)
- Config file (example 6 tests): `TransferBench examples/example.cfg` (compare each listed bandwidth against thresholds; Test 5 CPU-only has no GPU threshold)

Record: Save raw output and a summarized table of achieved vs required bandwidths in acceptance log.

RCCL AllReduce

Build tests:

```
git clone https://github.com/ROCm/rccl-tests.git
cd rccl_tests/
make NCCL_HOME=/opt/rocm/
```

Run:

```
build/all_reduce_perf -b 8 -e 8G -f 2 -g 8
```

Pass if bus bandwidth (large message, ~8 GB) ≥ 304 GB/s.

rocBLAS GEMM Benchmarks

For installation, review rocBLAS documentation:

- [Linux installation](#)
- [Windows installation](#)

Run each until peak (stable) TFLOPS observed. Capture highest achieved value.

FP32:

```
rocblas-bench -f gemm -r s -m 4000 -n 4000 -k 4000 --lda 4000 --ldb 4000 --ldc
```

Pass: $\geq 94,100$ TFLOPS.

BF16 (strided batched):

```
rocblas-bench -f gemm_strided_batched_ex --transposeA N --transposeB T -m 1024
```

Pass: $\geq 130,600$ TFLOPS.

INT8 (strided batched):

```
rocbblas-bench -f gemm_strided_batched_ex --transposeA N --transposeB T -m 1024
```

Pass: $\geq 162,700$ TFLOPS.

BabelStream

Build:

```
git clone https://github.com/UoB-HPC/BabelStream.git
cd BabelStream
cmake -Bbuild -H. -DMODEL=hip -DCMAKE_CXX_COMPILER=hipcc
cmake --build build
export PATH=$PWD/build:$PATH
```

Wrapper (example using MPI) to pin device per rank (`wrapper.sh`):

```
#!/bin/bash
hip-stream --device $OMPI_COMM_WORLD_RANK -n 50 -s 268435456
```

Run:

```
chmod u+x wrapper.sh
mpiexec -n 8 wrapper.sh
```

Pass thresholds (MB/s): Copy $\geq 4,177,285$; Mul $\geq 4,067,069$; Add $\geq 3,920,853$; Triad $\geq 3,885,301$; Dot $\geq 3,660,781$.

Recording & Reporting

For each benchmark capture:

1. Command line used (with environment variables if any).
2. Raw tool output (saved to log files).
3. Extracted key metric vs threshold (PASS/FAIL).
4. Date/time, system identifier, ROCm version, driver/firmware versions.

In automated pipelines, normalize outputs into a JSON summary similar to AGFHC results for archival and trend analysis.

AGFHC Validation

This section presents a comprehensive approach to validating AMD Instinct™ GPU systems using the AMD GPU Field Health Check (AGFHC) tool. It covers:

- Install and verify the AGFHC tool and its prerequisites
- Available test recipes and their functional coverage
- Execute validation recipes
- Interpret test results

AGFHC enables users to systematically assess the health and functionality of their systems by running a series of automated hardware and subsystem tests, grouped into targeted validation “recipes.”

AGFHC recipes are designed to exercise all major subsystem components—PCIe, HBM memory, compute engines, power, and more—efficiently identifying hardware or configuration issues. PASS/FAIL criteria are robustly enforced with detailed logging available for each run, supporting integration into broader automation and cluster management workflows.

For complete details, extended guidance, and troubleshooting tips, consult the *AGFHC User Guide*. The following sections step through installation, command usage, recipe selection, and result interpretation to help you confidently validate your AMD GPU cluster.

AGFHC Installation

Reach out to your AMD customer success team for specific installation steps regarding AGFHC.

The ROCm Validation suite (RVS) is a prerequisite of AGFHC. Make sure that this is installed as of the ROCm software installation. For example, on Ubuntu:

```
sudo apt install rocm-validation-suite
```

AGFHC Expected Output

The following is an example output for AGFHC with 1.23.3 version and with ROCm 7.0.0.

```
AGFHC tool is being installed... Please wait!
ROCm version 7.0.0 is detected.
Installing AGFHC packages ...
Installation successful!
Summary of installation:
Package Name Status Version-----
-----
agfhc Installed 1.24.1-----
firexs2 Installed 4.0.0.6-noble-----
transferbench Installed 1.62-6.2.0-66-----
rochpl Installed 7.0.2-1+836c282-----
minihpl Installed 0.1.0-8f5f175-----
Installation path: /opt/amd/agfhc
AGFHC Installation Succeeded!
```

For more information on installation, refer to *AGFHC User Guide*.

Verify AGFHC Tool Installation

To verify successful installation of the AGFHC tool, use the following steps.

Check Directory Contents

Navigate to the AGFHC installation directory and list its contents to ensure all required files and subdirectories are present.

```
ls /opt/amd/agfhc
```

Expected output (contents may vary slightly by version):

```
agfhc agfhc.py api_example.py hclib injects __pycache__ recipes system_monitor
```

This confirms the core AGFHC executables, supporting scripts, recipes, and modules are correctly installed.

Check the AGFHC Version

Run the AGFHC tool with the `-v` (version) flag to verify that it is accessible and to display its installed version and associated package details.

```
/opt/amd/agfhc/agfhc -v
```

Example output

```
agfhc version:  
1.24.1  
package: mi300x  
git version: 84d389f
```

Confirm that the version number and package correspond to the expected platform (e.g., "mi300x") and that the version matches what you intended to install.

Ensure the ROCm Validation Suite (RVS), which is a prerequisite for AGFHC, is installed:

```
dpkg -l | grep rocm-validation-suite # Ubuntu/Debian  
rpm -qa | grep rocm-validation-suite # RHEL/CentOS
```

If not present, install with:

```
sudo apt install rocm-validation-suite # Ubuntu/Debian  
sudo yum install rocm-validation-suite # RHEL/CentOS
```

Test Basic AGFHC Functionality

List available AGFHC recipes to confirm the CLI is fully functional:

```
/opt/amd/agfhc/agfhc --list
```

This should display a table of available tests and recipes.

AGFHC Command Options

AGFHC offers various test levels to suit different validation needs. Individual tests are grouped together as recipes to provide coverage in a functional area. To view available tests and recipes, as well as their details, use the following helpful commands.

- To view the available test and recipes:

```
/opt/amd/agfhc/agfhc --list
```

- To view extended information about a test:

```
/opt/amd/agfhc/agfhc -test-info <test>
```

- To view extended information about a recipe:

```
/opt/amd/agfhc/agfhc -recipe-info <recipe>
```

Validation AGFHC Recipe Tests

The tables below list the recommended and suggested AGFHC validation recipes along with their estimated run times for applicable products. These recipes are designed to comprehensively exercise and validate essential functional areas of AMD GPU-based systems. For additional details and test parameters, refer to the AGFHC User Guide.

Minimum Required AGFHC Tests

Recipe Name	Applicable Products	Iterations/Duration	Estimated Test Duration / Rationale
all_lvl5	All AMD MI3xx Instinct™ models	2 hours	Catches most errors
hbm_lvl5	All AMD MI3xx Instinct™ models	2 Hours, first iteration	Stress HBM
sleep 300 sec.		5 Minutes, second iteration	For silicon to contract to widen any cracks
hbm_lvl5	All AMD MI3xx Instinct™ models	2 Hours, second iteration	Stress HBM
sleep 300 sec.		5 Minutes, third iteration	For silicon to contract to widen any cracks
hbm_lvl5	All AMD MI3xx Instinct™ models	2 Hours, third iteration	Stress HBM
sleep 300 sec.		5 Minutes, fourth iteration	For silicon to contract to widen any cracks
hbm_lvl5	All AMD MI3xx Instinct™ models	2 Hours, fourth iteration	Stress HBM
sleep 300 sec.		5 Minutes, fifth iteration	For silicon to contract to widen any cracks
gfx_lvl4	All AMD MI3xx Instinct™ models	1 Hour	GPU stress test to hot spot test GPU needed for DLC systems
sleep 300 sec.		5 Minutes, sixth iteration	For silicon to contract to widen any cracks
minihpl	All AMD MI3xx Instinct™ models	3 Hours	Search for voltage failures and stress HBM

Recipe Name	Applicable Products	Iterations/Duration	Estimated Test Duration / Rationale
pcie_lvl2	All AMD MI3xx Instinct™ models	10 Minutes	Check for link degradation
Total			14 Hours and 40 Minutes

Recommended AGFHC Tests

Recipe Name	Applicable Products	Iterations/Duration	Estimated Test Duration / Rationale
all_lvl5	All AMD MI3xx Instinct™ models	2 hours	Catches most errors
hbm_lvl5	All AMD MI3xx Instinct™ models	2 Hours, first iteration	Stress HBM
sleep 300 sec.		5 Minutes, second iteration	For silicon to contract to widen any cracks
hbm_lvl5	All AMD MI3xx Instinct™ models	2 Hours, second iteration	Stress HBM
sleep 300 sec.		5 Minutes, third iteration	For silicon to contract to widen any cracks
hbm_lvl5	All AMD MI3xx Instinct™ models	2 Hours, third iteration	Stress HBM
sleep 300 sec.		5 Minutes, fourth iteration	For silicon to contract to widen any cracks
hbm_lvl5	All AMD MI3xx Instinct™ models	2 Hours, fourth iteration	Stress HBM
sleep 300 sec.		5 Minutes, fifth iteration	For silicon to contract to widen any cracks
gfx_lvl4	All AMD MI3xx Instinct™ models	1 Hour	GPU stress test to hot spot test GPU needed for DLC systems
sleep 300 sec.		5 Minutes, sixth iteration	For silicon to contract to widen any cracks
minihpl	All AMD MI3xx Instinct™ models	10 hours	10 Hours

Recipe Name	Applicable Products	Iterations/Duration	Estimated Test Duration / Rationale
xgmi_lvl1	All AMD MI3xx Instinct™ models	5 Minutes	Check for link degradation
pcie_lvl2	All AMD MI3xx Instinct™ models	10 Minutes	Check for link degradation
Total			20 Hours and 45 Minutes

AGFHC Recipe Coverage Details

A summary of each AGFHC validation recipe and its primary focus area are given below, along with a sample of the extended recipe information output generated by AGFHC tool. To access the extended information for a specific recipe, run the following command, replacing

`<recipe>` with the desired recipe name:

```
/opt/amd/agfhc/agfhc --recipe-info <recipe>
```

For comprehensive instructions, test scope, and result interpretation, refer to the AGFHC User Guide.

all_lvl5

High level test summary:

- **PCIe Subsystem:** Tests PCIe link status, speed, width, and stress bandwidth (host-to-device, device-to-host, and bidirectional).
- **Memory Subsystem:** Exercises and validates HBM (High Bandwidth Memory) through stress tests such as bandwidth, dual stream, and random access patterns.
- **Compute Subsystem:** Runs compute kernels at various data types and loads, verifying the stability and peak capability of the GPU compute units.
- **Power and Thermal:** Max power and sustained stress kernels help uncover errors that show up under load.

Extended information:

```

$ /opt/amd/agfhc/agfhc --recipe-info all_lvl5

Log Directory: /root/agfhc/logs/agfhc_20250707-082147
Name: all_lvl5
Title: A ~2h check across system
Path: /opt/amd/agfhc/recipes/mi350x/all_lvl5.yml
Contents:
Test Title Mode Approximate Time
pcie_link_status PCIe Link Status 1 iteration 0:00:08
pcie_d2h_peak PCIe BW D2H Peak 1 iteration 0:00:14
pcie_h2d_peak PCIe BW H2D Peak 1 iteration 0:00:23
dma_h2d_peak DMA BW H2D Peak 1 iteration 0:00:23
dma_d2h_peak DMA BW D2H Peak 1 iteration 0:00:23
dma_bidi_peak DMA BW BiDi Peak 1 iteration 0:00:23
xgmi_a2a XGMI BW A2A 1 iteration 0:10:09
hbm_s16 HBM S16 1 iteration 0:10:15
gfx_maxpower GFX Max Power 1 iteration 0:10:05
hbm_ds HBM Dual Stream 1 iteration 0:05:10
hbm_remix2 HBM Remix2 1 iteration 0:05:20
hbm_s16_ds HBM S16 Dual Stream 1 iteration 0:05:15
hbm_s16 HBM S16 1 iteration 0:05:15
mall MALL 1 iteration 0:05:35
hbm_bw HBM BW 1 iteration 0:01:05
gfx_bf16ri GFX bf16 rand int 1 iteration 0:01:20
gfx_bf16tf GFX bf16 trig float 1 iteration 0:01:20
gfx_fp16ri GFX fp16 rand int 1 iteration 0:01:20
gfx_fp16tf GFX fp16 trig float 1 iteration 0:01:20
gfx_fp8ri GFX fp8 rand int 1 iteration 0:01:20
gfx_fp8tf GFX fp8 trig float 1 iteration 0:01:20
gfx_fp8ri_e4m3 GFX fp8 rand int e4m3 1 iteration 0:01:15
gfx_fp8ri_e5m2 GFX fp8 rand int e5m2 1 iteration 0:01:15
gfx_fp8tf_e4m3 GFX fp8 trig float e4m3 1 iteration 0:01:15
gfx_fp8tf_e5m2 GFX fp8 trig float e5m2 1 iteration 0:01:15
athub ATHUB 1 iteration 0:05:05
hbm_ds_ntd HBM Dual Stream NTD 1 iteration 0:25:10
sprites SPRITES 1 iteration 0:25:29
-----
Total: 02:08:47
Summary:
Tests: 0 Total, 0 Executed, 0 Skipped
Total Time: 00:00:03
Log directory: /root/agfhc/logs/agfhc_20250707-082147
Program exiting with return code AGFHC_SUCCESS [0]

```

hbm_lvl5

Note

This test should be run twice to better exercise the HBM memory ensuring no ECC exceptions are present.

- **HBM Bandwidth:** Measures and stresses memory read/write throughput.
- **HBM Data Patterns:** Performs wide pattern tests (dual stream, single/dual stream random, and sequential).
- **Memory Error Detection:** Looks for correctable/uncorrectable errors under load—useful for catching early DIMM or silicon issues.

Extended information

```
$ /opt/amd/agfhc/agfhc --recipe-info hbm_lvl5

Log Directory: /root/agfhc/logs/agfhc_20250707-082635
Name: hbm_lvl5
Title: A ~2h HBM workload
Path: /opt/amd/agfhc/recipes/mi350x/hbm_lvl5.yml
Contents:
Test Title Mode Approximate Time
hbm_bw HBM BW 1 iteration 0:01:05
hbm_remix2 HBM Remix2 1 iteration 0:29:30
hbm_ds HBM Dual Stream 1 iteration 0:29:20
hbm_s16_ds HBM S16 Dual Stream 1 iteration 0:29:25
hbm_s16 HBM S16 1 iteration 0:29:25
hbm_bw HBM BW 1 iteration 0:01:05
-----
Total: 01:59:50
Summary:
Tests: 0 Total, 0 Executed, 0 Skipped
Total Time: 00:00:03
Log directory: /root/agfhc/logs/agfhc_20250707-082635
Program exiting with return code AGFHC_SUCCESS [0]
```

gfx_lvl4

Note

This test should be run for at least one hour to ensure no issues with the cooling system. Thermal testing the HBM is covered with the HBM tests.

- **GPU thermal stress test:** Executes a GFX matrix workload that combines precision-specific math (FP8/FP16/BF16/FP32/FP64) and transcendental functions. Doing so generates hot spots on the GPU to test the thermal solution.
- **HBM thermal stress test:** Raw HBM bandwidth with dual concurrent streams (read/write or bidirectional), typically using non-temporal access patterns to bypass caches and hit memory controllers directly.
- **GPU max power test:** Maximum board power by combining high-utilization compute and memory traffic to exercise SMU power management, VRMs, and thermal solution.
- **Holistic workloads:** Executing workloads intended to stress both the GPU matrix and the HBM functioning as a graphics sanity check.

Extended information

```
$ /opt/amd/agfhc/agfhc --recipe-info gfx_lvl4

Log Directory: /home/user/agfhc/logs/agfhc_20260325-151748
Name: gfx_lvl4
Title: A ~1h GFX workload
Path: /opt/amd/agfhc/recipes/mi300x/gfx_lvl4.yml
Contents:
Test Title Mode Approximate Time
gfx_dgemm GFX dgemm 1 iteration 0:01:30
gfx_sgemm GFX sgemm 1 iteration 0:01:15
gfx_fp8ri GFX fp8 rand int 1 iteration 0:01:20
gfx_fp16ri GFX fp16 rand int 1 iteration 0:01:20
gfx_bf16ri GFX bf16 rand int 1 iteration 0:01:20
gfx_fp8tf GFX fp8 trig float 1 iteration 0:01:20
gfx_fp16tf GFX fp16 trig float 1 iteration 0:01:20
gfx_bf16tf GFX bf16 trig float 1 iteration 0:01:20
hbm_ds_ntd HBM Dual Stream NTD 1 iteration 0:01:10
gfx_maxpower GFX Max Power 1 iteration 0:10:05
rochpl rocHPL 2 iterations 0:15:40
sprites SPRITES 1 iteration 0:20:29
-----
Total: 00:58:09
Summary:
Tests: 0 Total, 0 Executed, 0 Skipped
Total Time: 00:00:05
Log directory: /home/user/agfhc/logs/agfhc_20260325-151748
Program exiting with return code AGFHC_SUCCESS [0]
```

xgmi_lvl1

- **XGMI Link Health:** Confirms all links are negotiated at expected width and speed.
- **XGMI Bandwidth:** Benchmarks GPU-to-GPU bandwidth, ensuring full performance.
- **Error Detection:** Looks for XGMI-specific errors that may disrupt sustained transfers.

Extended information

```
$ /opt/amd/agfhc/agfhc --recipe-info xgmi_lvl1

Log Directory: /home/user/agfhc/logs/agfhc_20250902-190619
Name: xgmi_lvl1
Title: A ~5m xGMI workload
Path: /opt/amd/agfhc/recipes/mi350x/xgmi_lvl1.yml
Contents:
Test Title Mode Approximate Time
xgmi_a2a XGMI BW A2A 1 iteration 0:05:09
-----
Total: 00:05:09
Summary:
Tests: 0 Total, 0 Executed, 0 Skipped
Total Time: 00:00:03
Log directory: /home/user/agfhc/logs/agfhc_20250902-190619
Program exiting with return code AGFHC_SUCCESS [0]
```

pcie_lvl2

- **PCIe Link Health:** Confirms all links are negotiated at expected width and speed.
- **PCIe Bandwidth:** Benchmarks device-to-host and host-to-device bandwidth, ensuring full performance.
- **Error Detection:** Looks for PCIe-specific errors that may disrupt sustained transfers.

Extended information

```
$ /opt/amd/agfhc/agfhc --recipe-info pcie_lvl2

Log Directory: /home/user/agfhc/logs/agfhc_20250902-190647
Name: pcie_lvl2
Title: A ~10m PCIe workload
Path: /opt/amd/agfhc/recipes/mi350x/pcie_lvl2.yml
Contents:
Test Title Mode Approximate Time
pcie_link_status PCIe Link Status 1 iteration 0:00:08
pcie_d2h_peak PCIe BW D2H Peak 1 iteration 0:05:13
pcie_h2d_peak PCIe BW H2D Peak 1 iteration 0:05:21
pcie_link_status PCIe Link Status 1 iteration 0:00:08
-----
Total: 00:10:50
Summary:
Tests: 0 Total, 0 Executed, 0 Skipped
Total Time: 00:00:03
Log directory: /home/user/agfhc/logs/agfhc_20250902-190647
Program exiting with return code AGFHC_SUCCESS [0]
```


miniHPL

Applicable for all AMD Instinct™ MI3xx models:

- **Compute + Fabric:** Runs Linpack-like workloads (High Performance Linpack, “HPL”) across all GPUs, stressing interconnect, DRAM, and compute together, like full scientific workloads.
- **System Integration:** This is as close as AGFHC gets to “whole system” stability verification.

Extended information

```
$ /opt/amd/agfhc/agfhc --test-info minihpl

Log Directory: /home/user/agfhc/logs/agfhc_20250902-190543
Name: minihpl
Description: Executes miniHPL Exerciser.
Recipes:
  single_pass
target:
  This test does not have targets

Performance Benchmarking

Summary:
  Tests: 0 Total, 0 Executed, 0 Skipped
  Total Time: 00:00:04
  Log directory: /home/user/agfhc/logs/agfhc_20250902-190543
  Program exiting with return code AGFHC_SUCCESS [0]
```

Running AGFHC Tests

This section provides the command required to run specific AGFHC validation recipes.

Before starting, create a directory to store the test outputs:

```
mkdir /tmp/agfhc_output
```

AGFHC Test Commands

Test Name	Command Line
all_lvl5	/opt/amd/agfhc/agfhc -r all_lvl5 -o /tmp/agfhc_output
sleep	sleep 300
hbm_lvl5	/opt/amd/agfhc/agfhc -r hbm_lvl5 -o /tmp/agfhc_output
sleep	sleep 300
hbm_lvl5	/opt/amd/agfhc/agfhc -r hbm_lvl5 -o /tmp/agfhc_output
sleep	sleep 300
hbm_lvl5	/opt/amd/agfhc/agfhc -r hbm_lvl5 -o /tmp/agfhc_output
sleep	sleep 300
hbm_lvl5	/opt/amd/agfhc/agfhc -r hbm_lvl5 -o /tmp/agfhc_output
sleep	sleep 300
gfx_lvl4	/opt/amd/agfhc/agfhc -r gfx_lvl4 -o /tmp/agfhc_output
sleep	sleep 300
miniHPL	/opt/amd/agfhc/agfhc -t minihpl:d=3h -o /tmp/agfhc_output
xgmi_lvl1	/opt/amd/agfhc/agfhc -r xgmi_lvl1 -o /tmp/agfhc_output
pcie_lvl2	/opt/amd/agfhc/agfhc -r pcie_lvl2 -o /tmp/agfhc_output

Evaluating AGFHC Test Results

There are several methods for determining results of the tests.

Terminal Summary Output

After running a recipe test, AGFHC prints a summary to the terminal.

- **PASS:** If all tests in the recipe succeed, you will see a clear PASS or AGFHC_SUCCESS [0] message.
- **FAIL:** If any test fails, you will see a FAIL message or error an error code, and failed tests will be listed.

Results File

Each run generates detailed logs and a summary JSON file (typically named results.json) in the specified output directory. For example:

```
{
  "recipe": "all_lv15",
  "overall_result": "PASS",
  "tests": [
    {"name": "pcie_link_status", "result": "PASS"},
    {"name": "hbm_bw", "result": "PASS"},
    // ... additional test entries ...
  ]
}
```

If any result entry shows **FAIL**, that test did not pass.

Return Code

AGFHC ends with a return code:

- 0: All tests passed
- Non-zero: One or more tests failed

This is useful if integrating AGFHC into scripts or CI pipelines.

Running AGFHC with the Cluster Validation Suite

AGFHC tests recommended for deployment can be automated using the [Cluster Validation Suite](#).

Before running CVS, ensure that the configuration files for your specific platform and the nodes under test are correctly defined in the `cvs/input/config_file/` subdirectories.

After [installing the test suite](#), deploy AGFHC to the cluster using the following command:

```
pytest -vvv --log-file=/tmp/test.log -s ./tests/health/install/install_agfhc.py \
  --cluster_file input/cluster_file/cluster.json \
  --config_file input/config_file/health/mi300_health_config.json \
  --html=/var/www/html/cvs/agfhc.html --capture=tee-sys --self-contained-html
```

Once AGFHC is installed, run the deployment test set using:

```
pytest -vvv --log-file=/tmp/test.log -s ./tests/health/csp_qual_agfhc.py \
  --cluster_file input/cluster_file/cluster.json \
  --config_file input/config_file/health/mi300_health_config.json \
  --html=/var/www/html/cvs/agfhc.html --capture=tee-sys --self-contained-html
```

Note that exact tests and durations in the `tests/health/csp_qual_agfhc.py` may differ from this document. Best practice is to run the longer of the two since extended test coverage will reduce the chance of failure after deployment

[Terms and Conditions](#)

[Privacy](#)

[Trademarks](#)

[Supply Chain Transparency](#)

[Fair and Open Competition](#)

[UK Tax Strategy](#)

[Cookie Policy](#)

[Cookie Settings](#)

© 2026 Advanced Micro Devices, Inc